DQN

一、实验题目

实现DQN算法

在 CartPole-v0 环境中实现DQN算法。最终算法性能的评判标准：以算法收敛的reward大小、收敛所需的样本数量给分。 reward越高（至少是180，最大是200）、收敛所需样本数量越少，分数越高。

二、实验内容

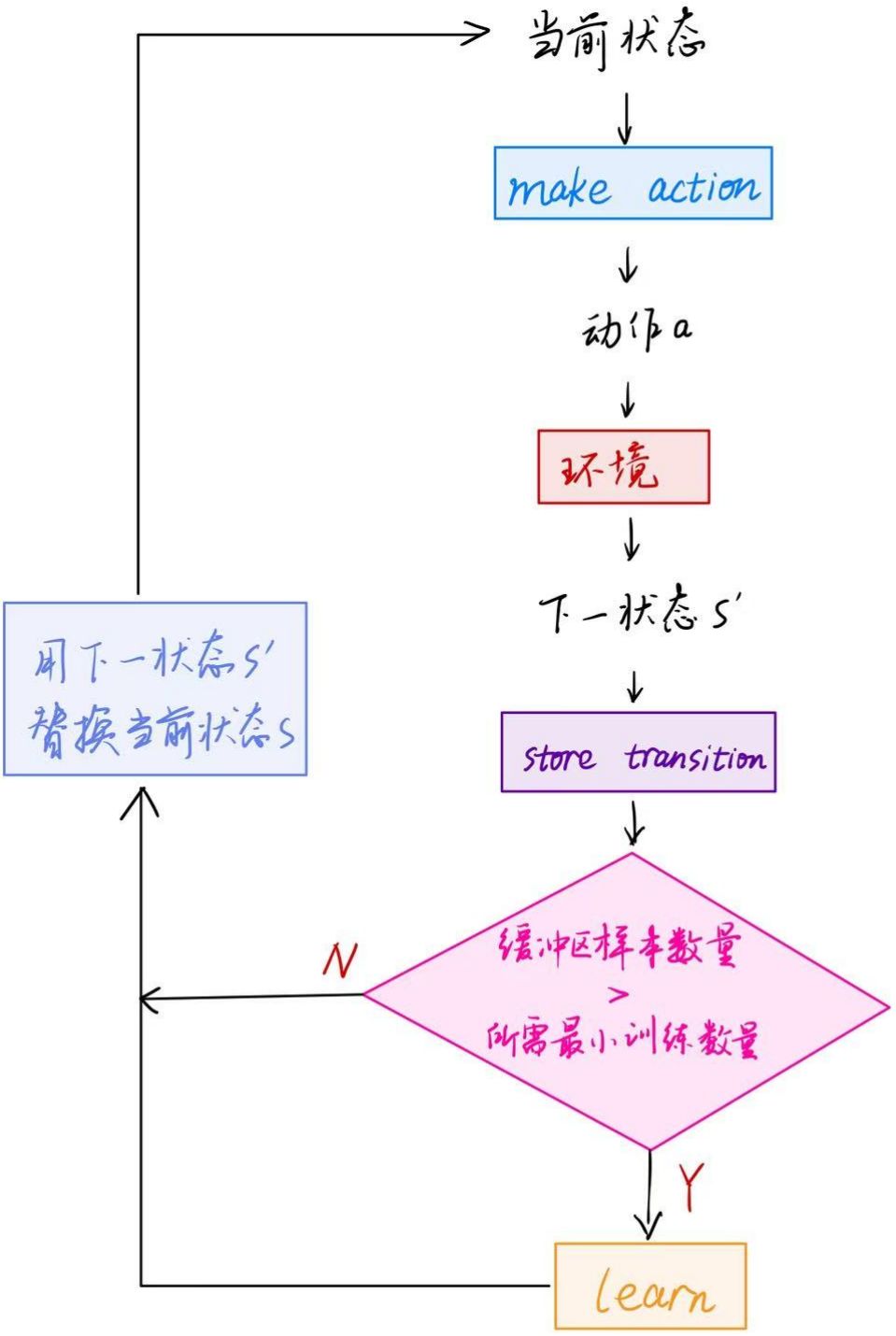
1.算法原理

1.1 基本介绍

DQN算法将Q_learning算法与深度学习相结合。与Q_learning算法不同，DQN使用深度神经网络来代替Q-Table，从而能够处理高维连续的状态和动作空间。其核心思想是通过神经网络拟合Q值函数，使得相近的状态得到相近的输出动作。

1.2 算法流程

为了方便说明，我画了以下流程图



2.关键代码展示

2.1 神经网络设计

```
# 网络结构
self.net = nn.Sequential(
    nn.Linear(input_size, hidden_size),
    nn.ReLU(),
    nn.LayerNorm(hidden_size),  # 添加层归一化
    nn.Linear(hidden_size, hidden_size),
    nn.ReLU(),
    nn.LayerNorm(hidden_size),
    nn.Linear(hidden_size, output_size)
)
```

2.2 动作选择

采用ε-贪婪策略

```
def make_action(self, observation, test=True):
    """
    根据当前状态选择动作 ，这里使用ε-贪婪策略
    Return predicted action of your agent
    Input:observation
    Return:action
    """
    # 确保observation是正确格式
    if not isinstance(observation, np.ndarray):
        observation = np.array(observation, dtype=np.float32)
    if len(observation.shape) > 1:
        observation = observation.flatten()

    if not test or np.random.random() < self.epsilon:
        # 探索：随机选择动作
        action = np.random.randint(self.action_dim)
    else:
        # 利用：选择Q值最大的动作
        with torch.no_grad():  # 测试时不需要计算梯度
            # 转换为张量并添加批次维度
            observation_tensor =
torch.FloatTensor(observation).unsqueeze(0).to(self.device)
            q_values = self.q_net(observation_tensor)
            action = q_values.argmax().item()
    return action
```

2.3 训练过程

```
    def train(self):
        for i in range(10):
            for episode in range(self.num_episodes//10):
                # 初始化本轮的回报
                episode_return=0

                # 重置环境（兼容新旧Gym版本）
                reset_result = self.env.reset()
                # 处理不同版本的返回结果
                if isinstance(reset_result, tuple) and len(reset_result) == 2:
                    # 新版本Gym (0.26+): (observation, info)
                    observation = reset_result[0]
                else:
                    # 旧版本Gym: 直接返回observation
                    observation = reset_result

                # 确保观测值是正确的一维数组
                if not isinstance(observation, np.ndarray):
                    observation = np.array(observation, dtype=np.float32)
                if len(observation.shape) > 1:
                    observation = observation.flatten()
```

```
                while True:
                    # 根据当前状态选动作
                    action =self.make_action(observation)

                    # 执行动作
                    step_result=self.env.step(action)
                    if len(step_result) == 4:
                        # 旧版本: observation, reward, done, info
                        next_observation, reward, done, _ = step_result
                        truncated = False
                    elif len(step_result) == 5:
                        # 新版本: observation, reward, done, truncated, info
                        next_observation, reward, done, truncated, info =
step_result
                    else:
                        # 默认处理
                        next_observation, reward, done, truncated =
step_result[0], step_result[1], step_result[
                            2], False

                    # 确保next_observation是正确格式，即一维数组
                    if not isinstance(next_observation, np.ndarray):
                        next_observation = np.array(next_observation,
dtype=np.float32)
                    if len(next_observation.shape) > 1:
                        next_observation = next_observation.flatten()

                    # 合并done和truncated（新版本特性）
                    is_done = done or truncated

                    # 将样本存入经验回放缓冲区
                    self.replay_buffer.push(observation, action, reward,
next_observation, is_done)

                    # 当缓冲区样本数足够时开始训练
                    if len(self.replay_buffer) > self.minimal_size:
                        # 从缓冲区采样mini-batch
                        states, actions, rewards, next_states, dones =
self.replay_buffer.sample(self.batch_size)
                        # 数据类型转换并加载到指定设备
                        states = torch.FloatTensor(states).to(self.device)
                        actions = torch.LongTensor(actions).view(-1,
1).to(self.device)

                        rewards = [r if not d else -10 for r, d in zip(rewards,
dones)]  # 失败时增加惩罚
                        rewards = torch.FloatTensor(rewards).view(-1,
1).to(self.device)
                        next_states =
torch.FloatTensor(next_states).to(self.device)
                        dones = torch.FloatTensor(dones).view(-1,
1).to(self.device)

                        # 计算当前状态-动作的Q值
                        q_values = self.q_net(states).gather(1, actions)
                        # 计算下一状态的最大Q值（使用目标网络）
```

```
                        max_next_q_values = self.target_q_net(next_states).max(1)
[0].view(-1, 1).detach()

                        # 计算TD目标值
                        q_targets = rewards + self.gamma * max_next_q_values * (1
- dones)
                        # 计算损失（均方误差）
                        loss = torch.mean(torch.nn.functional.mse_loss(q_values,
q_targets))

                        # 优化器梯度清零
                        self.optimizer.zero_grad()
                        # 反向传播计算梯度
                        loss.backward()
                        # 添加梯度裁剪 (防止梯度爆炸)
                        torch.nn.utils.clip_grad_norm_(self.q_net.parameters(),
max_norm=10)
                        # 更新网络参数
                        self.optimizer.step()

                        # 定期更新目标网络（硬更新）
                        if self.count % self.target_update == 0:

 self.target_q_net.load_state_dict(self.q_net.state_dict())

                        self.count += 1  # 更新计数器
                        # 更新探索率（设置下限防止完全失去探索能力）
                        self.epsilon = max(self.epsilon * 0.995, 0.01)

                    observation=next_observation  # 更新状态
                    episode_return+=reward  # 累加收益

                    if done or truncated:
                        break

                # 记录本轮回报
                self.return_list.append(episode_return)
                # 写入TensorBoard日志
                self.writer.add_scalar('Reward/Episode', episode_return,
len(self.return_list))

                # 每10轮episode输出一次平均回报
                if (episode + 1) % 10 == 0:
                    avg_return = np.mean(self.return_list[-10:])
                    print(
                        f"Episode {episode + 1:04d}, Average Return:
{avg_return:.2f}, Epsilon: {self.epsilon:.3f}")
```

3.创新点

3.1 在每层神经网络之间添加层归一化

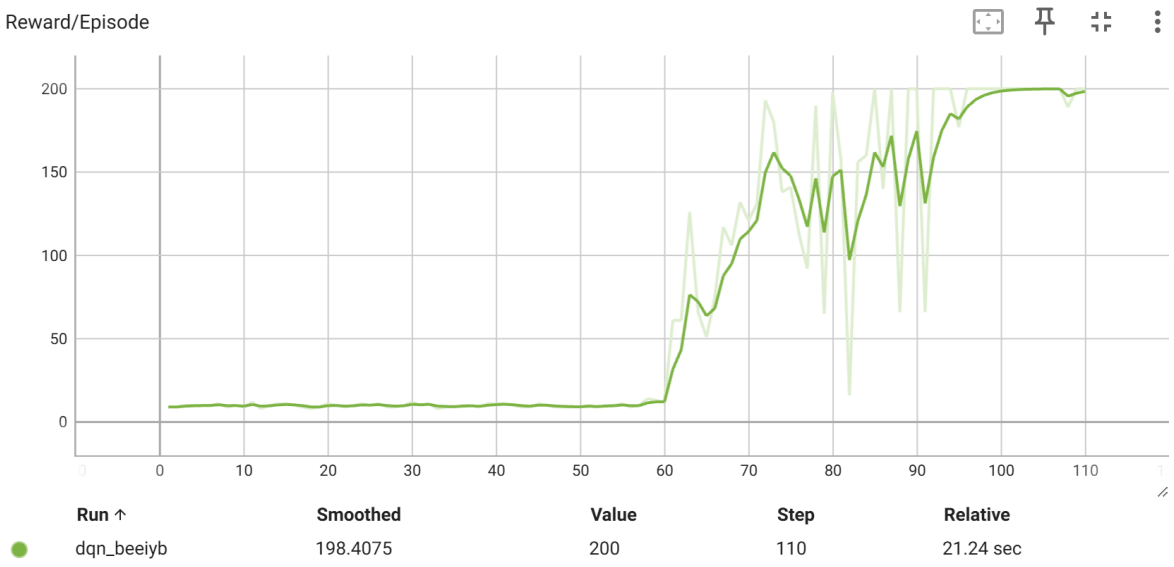
```
# 网络结构
self.net = nn.Sequential(
    nn.Linear(input_size, hidden_size),
    nn.ReLU(),
    nn.LayerNorm(hidden_size),  # 添加层归一化
    nn.Linear(hidden_size, hidden_size),
    nn.ReLU(),
    nn.LayerNorm(hidden_size), # 添加层归一化
    nn.Linear(hidden_size, output_size)
)
```

层归一化的优点有：

1. **稳定训练过程**：通过将每层的输入归一化为均值为0、方差为1的标准分布，减少了内部协变量偏移（Internal Covariate Shift）问题，使训练更稳定。
2. **缓解梯度问题**：有助于缓解梯度消失和梯度爆炸问题，使得梯度在反向传播时更稳定。
3. **加速收敛**：归一化操作使得损失函数曲面更平滑，可以使用更大的学习率，从而加速收敛。
4. **泛化能力强，适应不同样本**：对于不同样本的同一层神经元，层归一化独立地进行归一化，因此对小批量大小不敏感（与批量归一化相比）。

三、实验结果及分析

最终效果

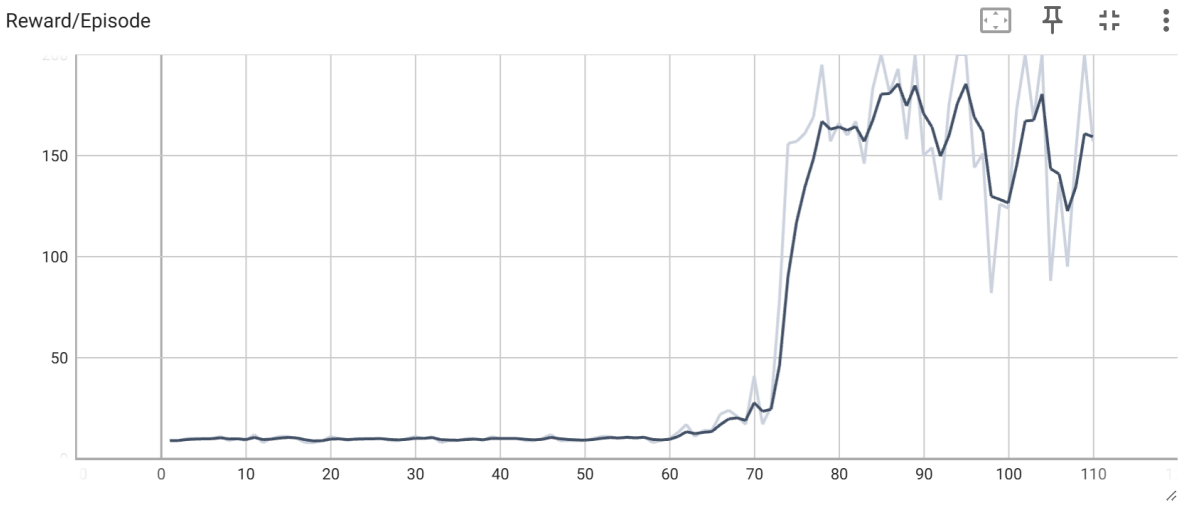


由图可见，大概只需100次迭代，reward就能达到200，之后尽管有小波动，也能保持在200左右。

有归一化与无归一化的对比

无归一化的收敛效果:

迭代110次的效果如下图所示。由图可见，收敛速度较快，但是后期的波动很大，及其不稳定，而且奖励达不到200

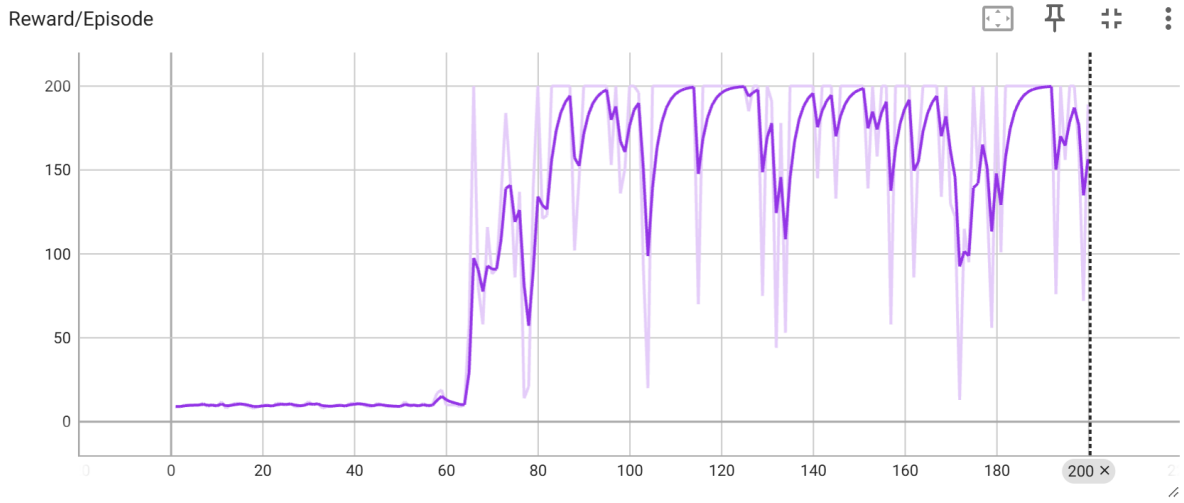


Run ↑	Smoothed	Value	Step	Relative
dqn_950dt5	159.2994	157	110	10.07 sec

起初，我以为是迭代次数太小的问题，于是我增加迭代次数到200次。

迭代200次的效果如下图所示。

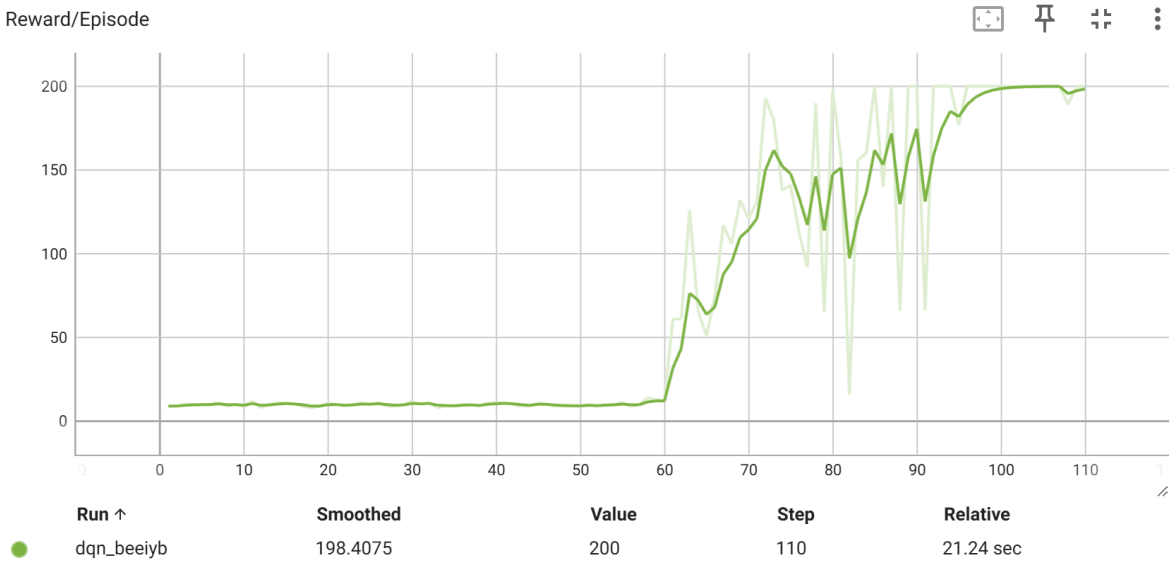
由图可见，虽然奖励可以达到200，且收敛速度较快，但是后期的波动很大，及其不稳定。



Run ↑	Smoothed	Value	Step	Relative
dqn_xyefr5	156.5026	189	200	1.054 min

有归一化的收敛效果

最终，我在每两层神经网络之间增加归一化，效果如下图所示。



由图可见，大概只需100次迭代，reward就能达到200，之后尽管有小波动，也能保持在200左右。此外，添加归一化后的收敛速度与没有使用归一化的收敛速度大致相同。

总结

通过在每层神经网络之间添加归一化，能够在保证收敛速度几乎不变的情况下，有效提高稳定性，可以达到教好的训练效果。

四、参考资料

[1] 初探强化学习(9)DQN详解-Deep Q Network保姆级别的分析(从Qlearning讲到DQN)_dqn网络结构-CSDN博客

[2] DQN基本概念和算法流程（附Pytorch代码） - 知乎